

KAVACH — Critical Limitations, Remediation Strategy & Part 2 Blueprint

A rigorous pre-implementation assessment of KAVACH v1.0 and the proposed Context-Aware Security Intelligence extension

Purpose. This document deliberately critiques KAVACH before the next implementation cycle. It identifies limitations in detection, policy, RAG, AI safety, impact analysis, evaluation, auditability, multilingual behavior and operational maturity. Each limitation is paired with a concrete remediation direction and a qualitative before/after project-quality score. Post scores are targets, not achieved results.

Current baseline. The existing project has previously demonstrated 164 passing tests, security precision/recall/F1 = 1.0/1.0/1.0 on its current benchmark, and impact precision ≈ 0.57 , recall ≈ 1.0 and F1 ≈ 0.719 . The RAG benchmark requires correction because relevant retrievals were observed while exact path comparison reported zero scores.

Executive Assessment

KAVACH is already a substantial applied engineering prototype. Its strength is the integration of security detection, policy enforcement, repository-aware RAG, evidence-grounded generation, change-impact analysis, validation, workflow tracing, APIs, frontend and CI security controls. The principal weaknesses are not that these components are useless; rather, several operate at a prototype boundary: known-format detection, small evaluation sets, moderate impact precision, limited semantic validation and incomplete protection of retrieved and generated context.

Part 2 should evolve the system rather than simply add features. The target is a **risk-adaptive, context-aware governance layer for AI-assisted software engineering** in which security is evaluated at developer input, repository ingestion, retrieved evidence, requested action, generated output and audit storage.

Target lifecycle: Developer Intent → Context-Aware Detection → Secret Detection → Risk Scoring → Action-Aware Policy → Secure RAG Retrieval → Untrusted-Context Protection → Impact Analysis → Evidence-Grounded Generation → Output Security Gate → Validation → Audit → CI Gate.

The implementation principle is to retain deterministic and explainable controls where they work well, and introduce semantic/ML reasoning only where rules are insufficient.

Before / Target-After Scorecard

These are qualitative engineering scores for prioritization, not measured benchmark values. A target score becomes evidence only after implementation and evaluation.

Limitation	Before	Target After	Priority
1. Rule-Based Sensitive-Data Detection and Unknown Identifiers	High	Very High	Critical
2. False Positives and Contextual Ambiguity	Medium-High	Very High	High
3. Limited Coverage of PII and Sensitive-Data Categories	High	Very High	Critical
4. Binary Decisions Need Risk-Adaptive and Action-Aware Policy	High	Very High	Critical
5. Sensitive Repository Context Can Leak Through RAG	High	Very High	Critical
6. Indirect Prompt Injection in Retrieved Repository Content	High	Very High	Critical
7. Secrets and Credentials Require a Dedicated Detector	High	Very High	Critical
8. Generated Output and Generated Code Need a Security Gate	High	Very High	Critical
9. Impact Analyzer Has Moderate Precision	Medium-High	High	High
10. RAG Evaluation Is Small and Has a Path-Normalization Failure	High	Very High	Critical
11. Security Evaluation Dataset Is Too Limited for Generalization Claims	Medium-High	Very High	High
12. Audit Logs Can Become a Secondary Leakage Channel	High	Very High	Critical
13. LLM Output Is Probabilistic and Validation Is Not Full Correctness	High	Very High	Critical
14. Authorization and Resource-Level Access Control Are Incomplete	Medium-High	High	High
15. Multilingual and Semantic Understanding Has Coverage Gaps	Medium-High	High	High
16. Production Scalability, Reliability and Operational Security	Medium	High	High
17. Over-Engineering and Feature-Creep Risk	High	Very High	Critical

1. Rule-Based Sensitive-Data Detection and Unknown Identifiers

Current limitation — what happens, why it happens, and why it matters

K

How KAVACH should fix it

A

What happens to the project after the fix

V

Project-quality score — Before: High **Target after successful fix:** Very High

2. False Positives and Contextual Ambiguity

Current limitation — what happens, why it happens, and why it matters

A

How KAVACH should fix it

What happens to the project after the fix

p

Project-quality score — Before: Medium-High **Target after successful fix:** Very High

3. Limited Coverage of PII and Sensitive-Data Categories

Current limitation — what happens, why it happens, and why it matters

T

How KAVACH should fix it

h

What happens to the project after the fix

e

Project-quality score — Before: High Target after successful fix: Very High

4. Binary Decisions Need Risk-Adaptive and Action-Aware Policy

Current limitation — what happens, why it happens, and why it matters

S

How KAVACH should fix it

e

What happens to the project after the fix

n

Project-quality score — Before: High **Target after successful fix:** Very High

5. Sensitive Repository Context Can Leak Through RAG

Current limitation — what happens, why it happens, and why it matters

R

How KAVACH should fix it

e

What happens to the project after the fix

p

Project-quality score — Before: High **Target after successful fix:** Very High

Project-quality score — Before: High **Target after successful fix:** Very High

7. Secrets and Credentials Require a Dedicated Detector

Current limitation — what happens, why it happens, and why it matters

P

How KAVACH should fix it

I

What happens to the project after the fix

I

Project-quality score — Before: High Target after successful fix: Very High

8. Generated Output and Generated Code Need a Security Gate

Current limitation — what happens, why it happens, and why it matters

S

How KAVACH should fix it

e

What happens to the project after the fix

C

Project-quality score — Before: High **Target after successful fix:** Very High

9. Impact Analyzer Has Moderate Precision

Current limitation — what happens, why it happens, and why it matters

T

How KAVACH should fix it

h

What happens to the project after the fix

e

Project-quality score — Before: Medium-High **Target after successful fix:** High

10. RAG Evaluation Is Small and Has a Path-Normalization Failure

Current limitation — what happens, why it happens, and why it matters

T

How KAVACH should fix it

h

What happens to the project after the fix

e

Project-quality score — Before: High **Target after successful fix:** Very High

11. Security Evaluation Dataset Is Too Limited for Generalization Claims

Current limitation — what happens, why it happens, and why it matters

T

How KAVACH should fix it

h

What happens to the project after the fix

e

Project-quality score — **Before:** Medium-High **Target after successful fix:** Very High

12. Audit Logs Can Become a Secondary Leakage Channel

Current limitation — what happens, why it happens, and why it matters

A

How KAVACH should fix it

u

What happens to the project after the fix

d

Project-quality score — Before: High **Target after successful fix:** Very High

13. LLM Output Is Probabilistic and Validation Is Not Full Correctness

Current limitation — what happens, why it happens, and why it matters

E

How KAVACH should fix it

V

What happens to the project after the fix

i

Project-quality score — Before: High **Target after successful fix:** Very High

14. Authorization and Resource-Level Access Control Are Incomplete

Current limitation — what happens, why it happens, and why it matters

C

How KAVACH should fix it

O

What happens to the project after the fix

n

Project-quality score — **Before:** Medium-High **Target after successful fix:** High

15. Multilingual and Semantic Understanding Has Coverage Gaps

Current limitation — what happens, why it happens, and why it matters

M

How KAVACH should fix it

U

What happens to the project after the fix

I

Project-quality score — Before: Medium-High **Target after successful fix:** High

16. Production Scalability, Reliability and Operational Security

Current limitation — what happens, why it happens, and why it matters

T

How KAVACH should fix it

h

What happens to the project after the fix

e

Project-quality score — Before: Medium Target after successful fix: High

17. Over-Engineering and Feature-Creep Risk

Current limitation — what happens, why it happens, and why it matters

K

How KAVACH should fix it

A

What happens to the project after the fix

V

Project-quality score — Before: High **Target after successful fix:** Very High

Integrated Part 2 Architecture

Security Intelligence: deterministic patterns + contextual features + multilingual normalization + secret detection.

Risk and Policy: sensitivity + confidence + requested action + resource scope + exposure destination → ALLOW / REVIEW / BLOCK / REDACT / REMEDIATE.

Secure RAG: scan at ingestion → attach security metadata → retrieve by relevance + policy → minimize/redact → treat context as untrusted data → final context security check.

Impact Intelligence: dependency graph + changed symbols + references + tests + semantic signals → ranked impacted components with reasons/confidence.

Generation Security: LLM proposal → output sensitivity/secret scan → code-security checks → tests → policy/impact validation → governed result.

Audit and Evaluation: privacy-preserving event records + normalized paths + held-out benchmarks + regression comparisons.

Implementation Order and Guardrails

1. Preserve the cleaned Phase 1–9 project as the v1.0 baseline. 2. Fix RAG benchmark path normalization before publishing RAG metrics. 3. Implement contextual detection and hard-negative evaluation. 4. Add risk scoring and action-aware policy. 5. Add secret detection and audit redaction. 6. Add security metadata and policy-aware RAG filtering. 7. Add repository prompt-injection isolation. 8. Add output security and stronger validation. 9. Improve impact ranking. 10. Add resource-policy abstractions. 11. Expand multilingual evaluation. 12. Re-run all old and new tests, security, impact, RAG and CI checks. 13. Update frontend, Swagger and documentation only after behavior is stable.

Non-negotiable regression rule: the original v1.0 tests should remain passing unless a behavior change is deliberate, documented and accompanied by updated tests. Every Part 2 feature must map to a limitation, have a test, and have an observable evaluation result.

Final Critical Verdict

KAVACH does not need to hide its limitations. The strongest project narrative is to show that limitations were measured, analyzed and converted into an engineering roadmap. The most valuable upgrades are contextual sensitive-data detection, risk-adaptive policy, security-aware RAG, secret detection, prompt-injection resistance, output security, privacy-preserving audit, robust evaluation and ranked impact analysis.

If implemented and evaluated successfully, KAVACH's claim becomes stronger and more precise: **a security-governed agentic software-engineering platform in which developer intent, repository evidence, generated artifacts and downstream change impact are treated as security-relevant boundaries**. The system should still be described as an academic/applied prototype unless production controls are actually implemented.

Important: all "Target After" scores in this document are goals. They must not be presented as achieved performance until the corresponding implementation and evaluation are completed.